

MCAST - BSc (Hons) Digital Games Development

ITDGM-606-2310 – Advanced Game Programming

Teamwork -> Deadline: 16th January 2026

Individual -> Deadline: 30th January 2026

General Guidelines:

- This assignment carries **75%** of the global mark.
- Read the scenario carefully and follow the instructions given
- Any reference from class notes, books or internet should be included in the appendix
- **Plagiarism** is strictly prohibited and will be penalised in accordance with the college's disciplinary procedures.
- Your lecturer will provide you with details to upload your work.
- Late assignments are not accepted
- Read carefully the rubrics

Section A - Teamwork (54%)

You have been employed as an external game development team by a local company.

Your task is to design, implement, and test a 3D game using **Unreal Engine 5.6**. You may choose to develop either a first-person or third-person game.

You are not allowed to use ready-made scripts unless approved by your lecturer.

You are also not allowed to use Blueprint-only implementations: **C++ must be used** for the core game implementation, as shown in class.

Teamwork Regulations

- Every team must have a private distributed version control system. You may use Perforce or Github.
- Every team member must commit regularly his/her work with appropriate comments.
- Group members are assigned by the lecturer. Groups can consist of two or three members only.
- Every student is responsible for a particular game level and functionalities as described in Table 1. Example: Student 1 must conduct the functionalities found under the column with header "Student 1". Team members cannot swap functionalities but code can be shared among project.
- Individual contributions will be assessed separately and can be awarded different grades. Students will be held individually accountable for their contribution to the project.
- **Marks are allocated for how well the team members collaborated.**
- Group members should notify the lecturer if any team member is not contributing. Students who do not contribute may be removed from the team by the lecturer, assigned to complete the project individually, and have their marks deducted accordingly.

Task 1

You must create a first-person or third-person game in a genre of your choice.

Evaluation will primarily focus on the technical implementation of your game, as described below.

High-quality textures, models, sounds, and level design are encouraged, but these should serve mainly to demonstrate the game's functionalities rather than being the primary focus.

All core gameplay mechanics and essential features must be implemented in C++.

The use of Unreal Engine Blueprints should be limited to minor or non-core elements.

Focus on writing clean, well-structured, and well-documented code.

Your game must include all the functionalities specified in Table 1.

Table 1: Functionalities

<i>A&A2 Apply a data-driven approach to gameplay components and logic</i> <i>E&C1 Create a fully functional gameplay system that demonstrates advanced technical design and implementation within a game context</i> <i>E&C2 Design and implement a game architecture that clearly separates data definitions from processing logic</i> <i>E&C3 Build memory management and optimization strategies to significantly improve the runtime performance of a game</i> <i>E&C4 Refactor and optimize the game codebase to enhance stability, scalability, and maintainability while preserving functionality</i>				
Functionality ID	Student 1 - Level 1	Student 2 - Level 2	Student 3 – Level 3 (if group has three students)	Marks
1	Design a third-person or first-person game and a high-quality map for your game. It should feature a clear game objective. The level must be different from the other students.	Design a third-person or first-person game and a high-quality map for your game. It should feature a clear game objective. The level must be different from the other students.	Design a third-person or first-person game and a high-quality map for your game. It should feature a clear game objective. The level must be different from the other students.	5
2	Design and implement a horizontal game system, for example item collection, ability upgrade or AI enemy movements. Each student specializes in one and only one system that other students then integrate.	Design and implement a horizontal game system, for example item collection, ability upgrade or AI enemy movements. Each student specializes in one and only one system that other students then integrate.	Design and implement a horizontal game system, for example item collection, ability upgrade or AI enemy movements. Each student specializes in one and only one system that other students then integrate.	8

3	The player can collect items that provide various benefits, such as power-ups or collectibles. This functionality may use one of the horizontal game systems developed in functionality 2.	The player can collect items that provide various benefits, such as power-ups or collectibles. This functionality may use one of the horizontal game systems developed in functionality 2.	The player can collect items that provide various benefits, such as power-ups or collectibles. This functionality may use one of the horizontal game systems developed in functionality 2.	6
4	AI-controlled enemies are required to patrol designated areas, chase the player upon detection, and engage in combat to reduce the player's health. This functionality may use one of the horizontal game systems developed in functionality 2.	AI-controlled enemies are required to patrol designated areas, chase the player upon detection, and engage in combat to reduce the player's health. This functionality may use one of the horizontal game systems developed in functionality 2.	AI-controlled enemies are required to patrol designated areas, chase the player upon detection, and engage in combat to reduce the player's health. This functionality may use one of the horizontal game systems developed in functionality 2.	6
5	Players receive items as rewards for eliminating enemies.	Players receive items as rewards for eliminating enemies.	Players receive items as rewards for eliminating enemies.	6
6	The player can view their health, current game objectives and activated abilities in a separate menu. The menu can be toggled on or off by pressing a specific key on the keyboard.	The player can view their health, current game objectives and activated abilities in a separate menu. The menu can be toggled on or off by pressing a specific key on the keyboard.	The player can view their health, current game objectives and activated abilities in a separate menu. The menu can be toggled on or off by pressing a specific key on the keyboard.	6
7	Integrate the levels and game systems developed in one single vertical slice of the game and generate coherent a game build.	Integrate the levels and game systems developed in one single vertical slice of the game and generate coherent a game build.	Integrate the levels and game systems developed in one single vertical slice of the game and generate coherent a game build.	5
8	Make use of a versioning system.	Make use of a versioning system.	Make use of a versioning system. Show the commits	5

	Show the commits with screenshots. Performance is highly recommended but not mandatory.	Show the commits with screenshots. Performance is highly recommended but not mandatory.	with screenshots. Performance is highly recommended but not mandatory.	
--	---	---	--	--

Task 2

As a team, prepare a detailed report in Microsoft Word describing the horizontal game systems implemented (as outlined in **Functionality ID 2** in Table 1) and explaining how these systems integrate and interact with each other within the game.

Your report must clearly address the following points:

- 1) Coding and design challenges.
- 2) Strengths and weaknesses in terms of performance and scalability.
- 3) Strengths and weaknesses in terms of maintenance.
- 4) A YouTube trailer showcasing your game and the main gameplay features developed.

A&A5 Implement targeted modifications to the codebase to address performance bottlenecks and architectural weaknesses affecting maintainability. [7 Marks]				
	PROFICIENT [7 Marks]	NEARING PROFICIENT [5-3 Marks]	NOVICE [1-2 Marks]	INADEQUATE [0 MARKS]
[7 Marks]	Provides a detailed and well-structured discussion of coding challenges, scalability, and maintenance aspects. The YouTube trailer is well-produced, clearly demonstrating the game's functionality and quality.	Discusses all required aspects but with limited depth or clarity. The trailer demonstrates minimal effort or lacks polish.	Provides only a basic or incomplete discussion of the required aspects. The trailer is missing or poorly executed.	Work is missing, incomplete, or irrelevant to the task

Section B - Individual (21%)

Task 1

The goal of this task is to demonstrate your ability to implement, debug and test a technical sound and well implemented game using Unreal Engine 5.6 with C++ and Blueprints.

You are required to design and implement a platformer set within a small map that showcases a variety of interesting gameplay interactions, such as area triggers, shifting platforms, moving obstacles, etc.

In addition, you must document your debugging and testing process in a Word document.

The documentation should include:

Debugging and Testing Report

- Clearly describe your debugging strategies and testing procedures.
- Include code snippets, screenshots, and explanations of issues identified and resolved.

Custom Gameplay Debugger

- Design and implement a custom Unreal Gameplay Debugger for your game system.
- It must visualize at least five (5) different variables relevant to gameplay
- Capture a screenshot of the viewport showing the debugger and attach it to your Word document.

Visual Logger Integration

- Implement and visualize with debug shapes at least five (5) different variables of the player-controlled pawn in the Unreal Visual Logger.
- Export the Visual Logger report and name the file:
Section_B_Task_1_{name}_{surname}.

Functional Tests

- Design and conduct five (5) different functional tests that evaluate the correctness and stability of key gameplay systems.
- Report your test results and include relevant output or screenshots in your documentation

A&A1 Analyze the structure and flow of a gameplay system to identify areas for refinement or improvement. A&A3 Experiment with code and asset configurations to assess their impact on memory usage and runtime efficiency A&A4 Apply debugging and testing techniques to identify and resolve issues within a game application.			
	PROFICIENT [15-21 Marks]	NEARING PROFICIENT [5-14 Marks]	NOVICE [0-4 Marks]
[21 Marks]	The game is technically well-implemented with clean, efficient C++ code. All debugging tools and test frameworks are correctly designed and documented. The Word report is detailed, well-structured, and includes relevant code snippets, screenshots, and clear discussions of technical challenges and solutions.	The game demonstrates partial implementation of debugging and testing tools. Some aspects may lack depth or clarity. The Word document is adequate but may contain missing details, unclear explanations, or limited evidence of testing.	Code is missing or poorly implemented. Documentation is minimal or of low quality. No evidence of proper debugging, testing, or technical analysis is provided.

--END OF ASSIGNMENT--